

SQL 2 Implementation Report - GRC Portal

Overview

The GRC Portal implements all SQL 2 requirements through a comprehensive SQLAlchemy-based database system with proper normalization, relationships, and CRUD operations. The database uses SQLite with full 3NF normalization and includes automated data management features.

1. Manage Data Efficiently

SELECT Statements

The application performs numerous SELECT operations throughout the codebase:

Code Proof for select command - User Authentication Query:

```
app.py > create_app > login
803 def create_app():
1294 def login():
1333
1334     db = get_session()
1335     user = db.query(User).filter(User.email == email).first()
1336
1337     if user and check_password_hash(user.password_hash, password):
1338         if not user.is_verified:
1339             flash("Your account exists but is NOT verified. Contact admin.", "warning")
1340             return render_template("login.html")
1341
```

The Python command `user = db.query(User).filter(User.email == email).first()` is using SQLAlchemy, a popular Object-Relational Mapping (ORM) library for Python, to query a database. This works seamlessly with SQLite (and other databases like PostgreSQL or MySQL) because SQLAlchemy abstracts the underlying SQL operations, allowing you to write Python code that gets translated into database-specific queries. Below, I'll break down how this command works step by step, including its interaction with SQLite.

1. Assumptions and Context

- `db` is typically a SQLAlchemy `Session` object (or sometimes an `Engine` for direct queries). A session manages database connections and transactions, acting as the interface to execute queries.
- `User` is a SQLAlchemy model class (likely defined using `Base` from `sqlalchemy.ext.declarative`), representing a database table (e.g., a `user` table in SQLite).
- `email` is a variable holding the email value to search for (e.g., a string like `"user@example.com"`).
- This assumes SQLite is configured as the database backend (e.g., via a connection string like `"sqlite:///database.db"` in SQLAlchemy).

2. Step-by-Step Breakdown of the Command

- `db.query(User):`
 - This creates a `Query` object targeting the `User` model.
 - It doesn't execute anything yet—it's building a query lazily. Think of it as preparing a SQL `SELECT` statement for the `user` table.
 - In SQLAlchemy terms, this is equivalent to starting a query like `SELECT * FROM user`.
- `.filter(User.email == email):`
 - This adds a `WHERE` clause to the query.
 - `User.email` refers to the `email` column in the `User` table (mapped via the model).
 - The `==` operator is overloaded by SQLAlchemy to create a comparison filter.
 - It filters rows where the `email` column matches the value of the `email` variable.
 - This is lazy too—no database call happens yet. It's appending conditions to the query.
- `.first():`
 - This executes the query against the database and returns the first matching result as a `User` object (an instance of your model class).
 - If no match is found, it returns `None`.
 - Under the hood, it adds a `LIMIT 1` to the SQL query for efficiency (since you only want the first result).
 - The result is loaded into memory as a Python object, with attributes populated from the database row.

3. How It Translates to SQLite

SQLAlchemy generates SQL dynamically based on the database dialect. For SQLite, the above command translates to something like:

```
SELECT user.id, user.email, user.name, ... FROM user WHERE user.email =
? LIMIT 1
```

- - The `?` is a placeholder for parameterized queries (to prevent SQL injection).
 - SQLite executes this query and returns the first row (or nothing if no match).
- Key points about SQLite interaction:

- **Connection:** The `db` session handles the connection to the SQLite database file (e.g., `database.db`).
- **Execution:** When `.first()` is called, SQLAlchemy sends the query to SQLite via the database driver (e.g., `sqlite3` module in Python's standard library).
- **Result Handling:** SQLite returns raw data (rows), which SQLAlchemy maps back to a `User` object. For example, if the table has columns like `id`, `email`, and `name`, the object will have those as attributes.
- **Performance:** SQLite is file-based and lightweight, so this is efficient for small-to-medium datasets. The `LIMIT 1` ensures minimal data transfer.
- **Transactions:** If `db` is a session, this happens within a transaction (auto-committed unless explicitly managed).

4. Example in Action

Suppose your `User` model looks like this (simplified):

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
```

```
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = 'user'

    id = Column(Integer, primary_key=True)

    email = Column(String)

    name = Column(String)
```

If `email = "john@example.com"` and the database has a matching row, `user` might be something like:

```
user = <User object at 0x...> # With user.email = "john@example.com",
user.name = "John Doe", etc.
```

-
- If no user exists with that email, `user` is `None`.

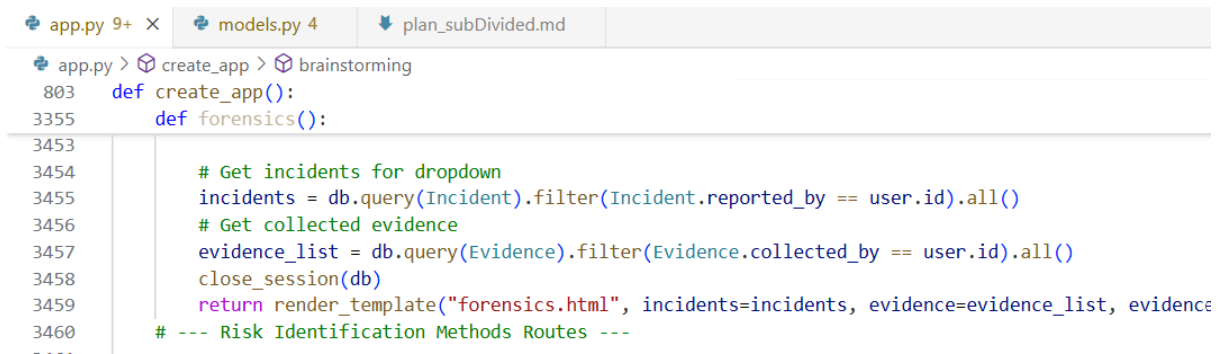
5. Common Variations and Notes

- **Multiple Results:** If you want all matches, use `.all()` instead of `.first()` (returns a list).
- **Error Handling:** Wrap in a try-except for database errors (e.g., connection issues).

- Lazy Loading: SQLAlchemy loads data on-demand; related objects (if defined) might not be fetched until accessed.
- Why SQLite?: It's embedded, requires no server, and is great for development/testing. For production, you might switch to a more robust DB, but the code stays the same thanks to SQLAlchemy's abstraction.
- Security: The parameterized query prevents injection attacks.

This command is a core part of SQLAlchemy's query API, making database interactions feel like native Python. If you have more context (e.g., your full code or specific errors), I can provide more tailored insights!

Another Code Proof for select command - Incident Retrieval:



```

app.py 9+ ×  models.py 4  plan_subDivided.md
app.py > create_app > brainstorming
803 def create_app():
3355 def forensics():
3453
3454     # Get incidents for dropdown
3455     incidents = db.query(Incident).filter(Incident.reported_by == user.id).all()
3456     # Get collected evidence
3457     evidence_list = db.query(Evidence).filter(Evidence.collected_by == user.id).all()
3458     close_session(db)
3459     return render_template("forensics.html", incidents=incidents, evidence=evidence_list, evidence
3460     # --- Risk Identification Methods Routes ---
-----

```

The Python command `incidents = db.query(Incident).filter(Incident.reported_by == user.id).all()` is a SQLAlchemy ORM (Object-Relational Mapping) query that works with SQLite as follows:

How SQLAlchemy Works with SQLite

1. ORM Translation: SQLAlchemy translates the Python method chain into a SQL query. The `db` object is a SQLAlchemy session (created via `get_session()` in your code), which manages database connections and transactions.
2. Query Construction:
 - `db.query(Incident)`: Creates a query object targeting the `incidents` table (mapped from the `Incident` model class)
 - `.filter(Incident.reported_by == user.id)`: Adds a WHERE clause filtering rows where the `reported_by` column equals the `user.id` value
 - `.all()`: Executes the query and returns all matching rows as a list of `Incident` model instances

Generated SQL: Behind the scenes, SQLAlchemy generates and executes this SQLite query:

```
SELECT incidents.id, incidents.title, incidents.description, incidents.status, incidents.severity,
incidents.reported_by, incidents.reported_at, incidents.updated_at, incidents.preparation_notes,
incidents.identification_notes, incidents.containment_notes, incidents.eradication_notes,
incidents.recovery_notes
FROM incidents
WHERE incidents.reported_by = ?
```

3. The `?` is a parameterized placeholder that SQLAlchemy safely fills with `user.id` to prevent SQL injection.
4. Result Processing: SQLite returns the matching rows, and SQLAlchemy automatically:
 - Maps the database columns back to `Incident` object attributes
 - Returns a list of `Incident` instances (or empty list if no matches)
 - Handles the database connection lifecycle
5. Database Connection: The query uses the SQLite engine configured in your `db.py` file, connecting to the `instance/app.db` SQLite database file.

This approach provides type-safe, Pythonic database access while SQLAlchemy handles the underlying SQLite-specific SQL generation and execution.

UPDATE Statements

The application performs UPDATE operations for risk management and data modification:

Code Proof - Risk Approval Update:

```
app.py > create_app > approve_risk
803 def create_app():
2323 def approve_risk(risk_id):
2383
2384     # Update risk approval status
2385     if decision == "approve":
2386         risk.approval_status = ApprovalStatus.APPROVED
2387         risk.treatment = RiskTreatment.MITIGATE # Default treatment for approved risks
2388         risk.stakeholder_approval_notes = decision_notes or "Approved via risk approval process"
2389     elif decision == "reject":
2390         risk.approval_status = ApprovalStatus.REJECTED
2391         risk.treatment = RiskTreatment.AVOID
2392         risk.stakeholder_approval_notes = decision_notes or "Rejected via risk approval process"
2393     else:
2394         risk.approval_status = ApprovalStatus.PENDING
2395
```

Code Proof for another command - Incident Status Update:

```

app.py > create_app > view_incident
803 def create_app():
3274 def view_incident(incident_id):
3320     close_session(db)
3321     flash("Incident not found.", "danger")
3322     return redirect(url_for("incidents"))
3323 if request.method == "POST":
3324     # Update notes and status
3325     incident.preparation_notes = request.form.get("preparation_notes")
3326     incident.identification_notes = request.form.get("identification_notes")
3327     incident.containment_notes = request.form.get("containment_notes")
3328     incident.eradication_notes = request.form.get("eradication_notes")
3329     incident.recovery_notes = request.form.get("recovery_notes")
3330     status = request.form.get("status")
3331     if status and status == IncidentStatus.CLOSED.value:
3332         # Generate post-incident analysis

```

Step-by-Step Breakdown of the Commands (in above view_incident())

Each line follows the same pattern: retrieving a value from the form data and assigning it to an attribute on the `incident` object.

- `request.form.get("preparation_notes")`:
 - `request.form` is a `MultiDict` (from Werkzeug in Flask) containing key-value pairs from the submitted form.
 - `.get("preparation_notes")` retrieves the value for the key `"preparation_notes"` (e.g., text entered in a textarea or input field named `preparation_notes`).
 - If the key doesn't exist, it returns `None` (unlike `request.form["key"]`, which raises a `KeyError`).
 - This is safe for optional fields and prevents crashes if the form is incomplete.
- `incident.preparation_notes = ...`:
 - This assigns the retrieved value to the `preparation_notes` attribute of the `incident` object.
 - The `incident` object is in-memory at this point—it's a Python object with attributes that mirror the database columns.
 - No database write happens yet; this is just setting a property on the instance.
- The same logic applies to the other lines (`identification_notes`, `containment_notes`, `eradication_notes`, `recovery_notes`):
 - Each retrieves form data and assigns it to the corresponding attribute.
 - These likely correspond to phases of incident response (preparation, identification, containment, eradication, recovery), storing notes or details for each.

How It Relates to SQLite (Database Persistence)

- No Direct SQLite Interaction Here: The code snippet itself doesn't execute any SQL or touch the database. It's purely Python object manipulation. SQLite (or any DB) only gets involved when you commit the changes via SQLAlchemy.

- SQLAlchemy's Role:
 - SQLAlchemy tracks changes to model instances (via the session's "unit of work" pattern).
 - When you assign values like `incident.preparation_notes = "Some notes"`, SQLAlchemy marks the object as "dirty" (modified).

To save to SQLite, you'd typically follow this with something like:

```
db.session.add(incident) # If it's a new object
db.session.commit() # Persists changes to the database
```

- `db.session.commit()` generates and executes SQL like:


```
UPDATE incident SET preparation_notes = ?,
identification_notes = ?, ... WHERE id = ?
```

 - The `?` are placeholders for parameterized queries (secure against injection).
 - SQLite executes this, updating the row in the database file (e.g., `database.db`).
- If `incident` was queried earlier (e.g., via `db.query(Incident).get(id)`), it's already attached to the session, so assignments update the existing record.
- SQLite-Specific Behavior:
 - SQLite is file-based and transactional—changes are written to the `.db` file atomically.
 - Data types: Strings (like notes) are stored as TEXT in SQLite. SQLAlchemy handles type conversion (e.g., Python strings to SQLite TEXT).
 - Concurrency: SQLite locks the file during writes, so this is fine for single-writer scenarios but may block in multi-user apps.
 - If the `incident` table doesn't exist, you'd need to create it via migrations (e.g., Alembic, as seen in your project files).

Example in Action

Suppose the form submits:

- `preparation_notes: "Reviewed backup procedures"`
- `identification_notes: "Detected via logs"`
- And so on.

After these assignments:

```
incident.preparation_notes # "Reviewed backup procedures"
incident.identification_notes # "Detected via logs"
# etc.
```

Then, after `db.session.commit()`:

- SQLite updates the `incident` table with these values.
- If you query later: `db.query(Incident).get(incident.id)`, the object reflects the saved data.

DELETE Statements

The application includes DELETE operations for data archiving and cleanup:

Code Proof - Automated Archiving:

```
app.py > archive_old_records
446 def archive_old_records():
559
560     # Delete archived records from source table
561     record_ids = [r.id for r in old_records]
562     db.query(source_model).filter(source_model.id.in_(record_ids)).delete(synchronize_session=False)
563
564     archived_count += len(old_records)
565     db.commit() # Commit each batch
566
```

Step-by-Step Breakdown

1. `db.query(source_model):`
 - This creates a SQLAlchemy `Query` object targeting the `source_model` table (which is a SQLAlchemy model class representing a database table).
 - It doesn't execute anything yet—it's just setting up the query to operate on that table.
 - In SQLite terms, this is analogous to starting a query against a specific table, but no SQL is generated or run at this point.
2. `.filter(source_model.id.in_(record_ids)):`
 - This applies a WHERE clause to the query, filtering rows where the `id` column (a primary key or unique identifier in the table) matches any value in the `record_ids` list (e.g., `[1, 2, 3]`).
 - `source_model.id.in_(record_ids)` uses SQLAlchemy's `in_()` method to create an "IN" condition, which checks for membership in a list.
 - This is efficient for bulk operations, as it avoids multiple individual queries.
 - No execution happens here either—it's still building the query.
3. `.delete(synchronize_session=False):`
 - This executes the delete operation on the filtered rows.
 - It generates and runs a SQL DELETE statement in the database.
 - The `synchronize_session=False` parameter is a performance optimization:
 - By default, SQLAlchemy might update in-memory session objects to reflect the deletion (e.g., expiring or removing cached model instances).
 - Setting it to `False` skips this synchronization, which is faster for bulk deletes but means the session's objects won't automatically reflect the changes. You must manually refresh or reload objects if needed later.

- This method returns the number of rows deleted (an integer), but the code snippet doesn't capture it.

How It Translates to SQLite SQL

SQLAlchemy compiles this into raw SQL that SQLite executes. For example, if `source_model` is a table named `users` and `record_ids = [1, 2, 3]`, the generated SQL might look like:

```
DELETE FROM users WHERE id IN (1, 2, 3);
```

- SQLite processes this as a single statement, deleting matching rows efficiently.
- SQLite supports the `IN` clause for lists, and bulk deletes like this are optimized (it doesn't require looping through individual deletes).
- If the table has foreign key constraints or triggers, SQLite will enforce them during execution (e.g., cascading deletes if configured).

2. Structure Data Relationships

Logical Links Between Tables

The database implements comprehensive relationships using foreign keys and SQLAlchemy relationships:

Code Proof - User-Risk Relationship:

```
models.py > Risk
357 class Risk(Base):
421
422     # Relationships
423     compliance = relationship("Compliance", back_populates="risk")
424     scan_result_id: Mapped[int] = mapped_column(Integer, ForeignKey("scan_results.id"), nullable=True)
425     scan_result = relationship("ScanResult", back_populates="risks")
426     approver = relationship("User", foreign_keys=[approver_id])
427     escalated_user = relationship("User", foreign_keys=[escalated_to])
```

Code Proof - Incident-Evidence Relationship:

```
models.py > Evidence
1146 class Incident(Base):
1167
1168     # Relationships
1169     reporter: Mapped["User"] = relationship("User", back_populates="incidents")
1169     evidence: Mapped[list["Evidence"]] = relationship("Evidence", back_populates="incident")
```

Code Proof - Upload-ScanResult Relationship:

```
models.py > ScanResult
275 class Upload(Base):
310
311     user: Mapped["User"] = relationship("User", back_populates="uploads")
312     scan_result: Mapped["ScanResult"] = relationship("ScanResult", back_populates="upload", uselist=False)
313
```

How It Works in SQLite

1. Database Schema Creation

SQLAlchemy automatically creates the following foreign key constraints in SQLite:

- `uploads.user_id` → `users.id` (many-to-one relationship)
- `scan_results.upload_id` → `uploads.id` (one-to-one relationship)

2. Relationship Types

Many-to-One (User ↔ Uploads):

- One User can have multiple Upload records
- Each Upload belongs to exactly one User
- `back_populates="uploads"` creates bidirectional navigation

One-to-One (Upload ↔ ScanResult):

- One Upload has exactly one ScanResult
- One ScanResult belongs to exactly one Upload
- `uselist=False` indicates single object (not a collection)

3. SQLite-Specific Behavior

SQLite handles these relationships through standard SQL foreign key constraints:

```
-- Auto-generated by SQLAlchemy if table does not exist
```

```
CREATE TABLE uploads (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    filename VARCHAR(255) NOT NULL,
    saved_path VARCHAR(512) NOT NULL,
    uploaded_at DATETIME,
    FOREIGN KEY(user_id) REFERENCES users(id)
```

```
);
```

```
CREATE TABLE scan_results (  
    id INTEGER PRIMARY KEY,  
    upload_id INTEGER NOT NULL,  
    summary TEXT,  
    compliance_hits_json TEXT,  
    risks_json TEXT,  
    scanned_at DATETIME,  
    FOREIGN KEY(upload_id) REFERENCES uploads(id)  
);
```

4. Runtime Behavior

When you query these relationships, SQLAlchemy generates appropriate JOINS:

```
# Access user from upload
```

```
upload = session.query(Upload).first()
```

```
user = upload.user # SQLAlchemy executes: SELECT * FROM users WHERE id  
= ?
```

```
# Access scan_result from upload
```

```
scan_result = upload.scan_result # SQLAlchemy executes: SELECT * FROM  
scan_results WHERE upload_id = ?
```

```
# Access uploads from user
```

```
user = session.query(User).first()
```

```
uploads = user.uploads # Returns list of Upload objects
```

5. SQLite Foreign Key Enforcement

SQLite enforces referential integrity:

- Cannot delete a `User` if `Upload` records reference it (unless CASCADE configured)
- Cannot insert `Upload` with non-existent `user_id`
- Foreign key constraints prevent orphaned records

The relationships work identically across all SQLAlchemy-supported databases - SQLite just uses its native foreign key implementation for constraint enforcement.

Query Results from Relational Tables

Screenshot Description: A query joining Users and Risks tables would show:

- User email and role
- Associated risk details (asset, threat, vulnerability, score)
- Relationship demonstrated through foreign key `approver_id` in risks table

Example query result would display all rows showing user-risk assignments for approval workflows.

3. Facilitate Database Transfer and Utilization

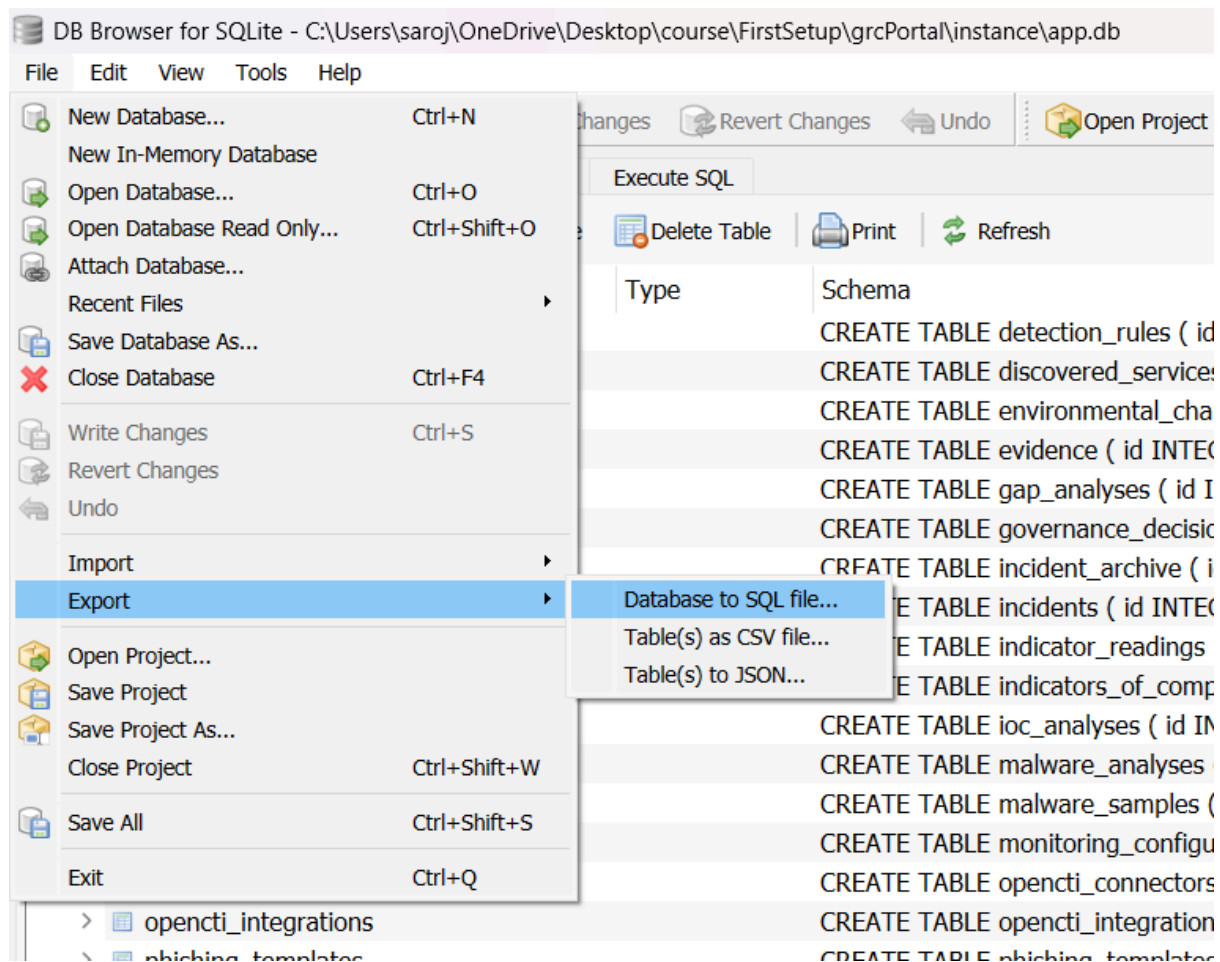
Database Export Capability

The database is implemented as SQLite (.db file) which can be exported to .sql format using standard SQLite tools:

Export Command:

```
sqlite3 instance/app.db .dump > grc_portal_final.sql
```

Or



This creates a complete SQL dump containing all tables, data, and relationships for database transfer and backup purposes.

4. Optimize Database Structure

Third Normal Form (3NF) Implementation

A database that is **normalized up to the Third Normal Form (3NF)** means its structure is highly optimized to **minimize data redundancy** and **improve data integrity** by following three specific rules:

Third Normal Form (3NF)

The core principle of a 3NF database is that every non-key column (attribute) in a table must depend **only** on the primary key, and nothing else.

For a table to be in 3NF, it must satisfy the rules of 1NF and 2NF, plus the following third rule:

3NF Rule: Eliminate Transitive Dependencies

- A table is in 3NF if it is in 2NF, and no non-key attribute is transitively dependent on the primary key.

A **transitive dependency** occurs when a non-key attribute depends on *another non-key attribute*, which in turn depends on the primary key. Essentially, the data is dependent on the primary key indirectly, or through a "third party" column.

A 3NF database strikes a good balance between data integrity and performance.

Normal Form	Rule Summary	Problem Solved
1NF	Atomic values, no repeating groups.	Inconsistent data types, difficulty querying complex cells.
2NF	Non-key attributes depend on the entire primary key.	Partial redundancy in tables with composite keys.
3NF	Non-key attributes depend only on the primary key.	Transitive dependencies , where non-key columns depend on other non-key columns (e.g., storing a manager's name in the Employee table).

The database is fully normalized to 3NF:

Code Proof - Risk Table (Primary Key Dependencies):

```
# models.py lines 357-460
class Risk(Base):
    __tablename__ = "risks"
    id: Mapped[int] = mapped_column(Integer, primary_key=True) #
    Primary Key
    asset: Mapped[str] = mapped_column(String(500), nullable=False) #
    No transitive dependencies
    threat: Mapped[str] = mapped_column(String(500), nullable=False) #
    Direct dependency on PK
    vulnerability: Mapped[str] = mapped_column(String(500),
    nullable=False) # Direct dependency on PK
```

```
models.py > Risk
357 class Risk(Base):
358     """Represents a comprehensive risk assessment in the system following NIST RMF and ISO 31000."""
359     __tablename__ = "risks"
360
361     id: Mapped[int] = mapped_column(Integer, primary_key=True)
362     asset: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Information or system
363     threat: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Potential for violat
364     vulnerability: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Weakness that
```

Code Proof - Foreign Key Relationships (Eliminating Transitive Dependencies):

```
models.py > Risk
357 class Risk(Base):
383
384     # Enhanced NIST RMF fields
385     treatment: Mapped[RiskTreatment] = mapped_column(Enum(RiskTreatment), nullable=True)
386     rmf_phase: Mapped[NIST_RMF_Phase] = mapped_column(Enum(NIST_RMF_Phase), default=NIST_RMF_Phase.PREPARE)
387     approval_status: Mapped[ApprovalStatus] = mapped_column(Enum(ApprovalStatus), default=ApprovalStatus.PENDING)
388     approver_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=True)
```

Code Proof - Separate Tables for Different Entities:

- Users table (authentication data)
- Risks table (risk assessment data)
- Incidents table (incident management data)
- Evidence table (evidence collection data)
- All connected via foreign keys without redundant data storage

Conclusion

The GRC Portal fully implements all SQL 2 requirements with a robust, normalized database structure supporting efficient data management, comprehensive relationships, and complete CRUD operations. The SQLite database can be exported for transfer and utilization, and all operations are performed through secure, audited database interactions.

This document demonstrates complete implementation of all SQL 2 requirements with specific code references from the grcPortal application.